



University of Science, VNU - HCM

Lab 05: 3D Object Transformation and Texture Mapping

Course: Computer Graphics

Course ID: CS411

Name: Le Tien Dat

Student ID: 23125028

Class: 23APCS2

Semester 2, 2025 - 2026

April 8, 2026.

Contents

1	Introduction	2
2	Rendering Default 3D Shapes	2
3	Plotting the Hyperboloid and Paraboloid	2
3.1	Paraboloid Core Algorithm	2
3.2	Hyperboloid Core Algorithm	3
4	Plotting Advanced Specific Shapes	3
5	Applying Clockwise Rotation	4
6	Texture Mapping Constraints	5
7	Conclusion	6

1 Introduction

This report summarizes the implementation details for the 3D Object Transformation and Texture Mapping. Building upon the baseline color cube and pyramid provided in the initial instructions, we developed a robust OpenGL pipeline rendering an 8-object grid. This grid showcases standard GLU primitives, complex parametric surfaces, independent per-face texturing, and isolated clockwise animations.

2 Rendering Default 3D Shapes

To render standard 3D quadrics consistently, we utilized the GLU library's `GLUquadric` objects alongside `freelut` primitives. This approach efficiently outputs the Sphere, Cylinder, and Cone. Because `gluCylinder` leaves the top and bottom faces completely hollow, we combined the quadric tube with `gluDisk` primitives placed at the local extremities to enforce solid, enclosed geometry suitable for independent texture mapping.

3 Plotting the Hyperboloid and Paraboloid

Since FreeGLUT and GLU do not natively support Hyperboloids of one sheet or Paraboloids, we constructed custom parametric generation functions using `GL_QUAD_STRIP`. We mathematically calculated the radius and heights at discrete stacks and slices, extracting exact spatial vertices, analytical surface normals (for accurate lighting/shading), and scaled UV coordinates.

3.1 Paraboloid Core Algorithm

```
void drawParaboloid(float baseRadius, float height, int slices, int stacks) {  
    for (int i = 0; i < stacks; i++) {  
        float y1 = (height * i) / stacks;  
        float y2 = (height * (i + 1)) / stacks;  
        float r1 = baseRadius * sqrt(1.0f - y1 / height);  
        float r2 = baseRadius * sqrt(1.0f - y2 / height);  
  
        glBegin(GL_QUAD_STRIP);  
        for (int j = 0; j <= slices; j++) {
```

```

        float theta = (2.0f * M_PI * j) / slices;
        // Analytical normals and vertices calculated here...
        glNormal3f(nx1, ny1, nz1);
        glTexCoord2f((float)j/slices, (float)i/stacks);
        glVertex3f(r1 * cos(theta), y1, r1 * sin(theta));
        // ... (Repeated for y2 and r2)
    }
    glEnd();
}
}

```

3.2 Hyperboloid Core Algorithm

```

void drawHyperboloid(float baseRadius, float waistRadius, float height,
                    int slices, int stacks) {
    float halfHeight = height / 2.0f;
    float coef = ((baseRadius * baseRadius) / (waistRadius * waistRadius) - 1.0f)
                / (halfHeight * halfHeight);

    for (int i = 0; i < stacks; i++) {
        float y1 = -halfHeight + (height * i) / stacks;
        float r1 = waistRadius * sqrt(1.0f + coef * y1 * y1);
        // ... evaluates geometry similarly via GL_QUAD_STRIP mappings
    }
}

```

4 Plotting Advanced Specific Shapes

For the Cube, rather than using `glutSolidCube` (which indiscriminately wraps one texture over the whole box), we implemented a custom `drawTexturedCube(float size, int* texIndices)` function. This explicitly draws six independent `GL_QUADS`, binding unique requested textures and resetting the texture coordinates (0,0) to (1,1) for every single face.

Here is the screen after plotting all the shapes (no texture mapping yet):

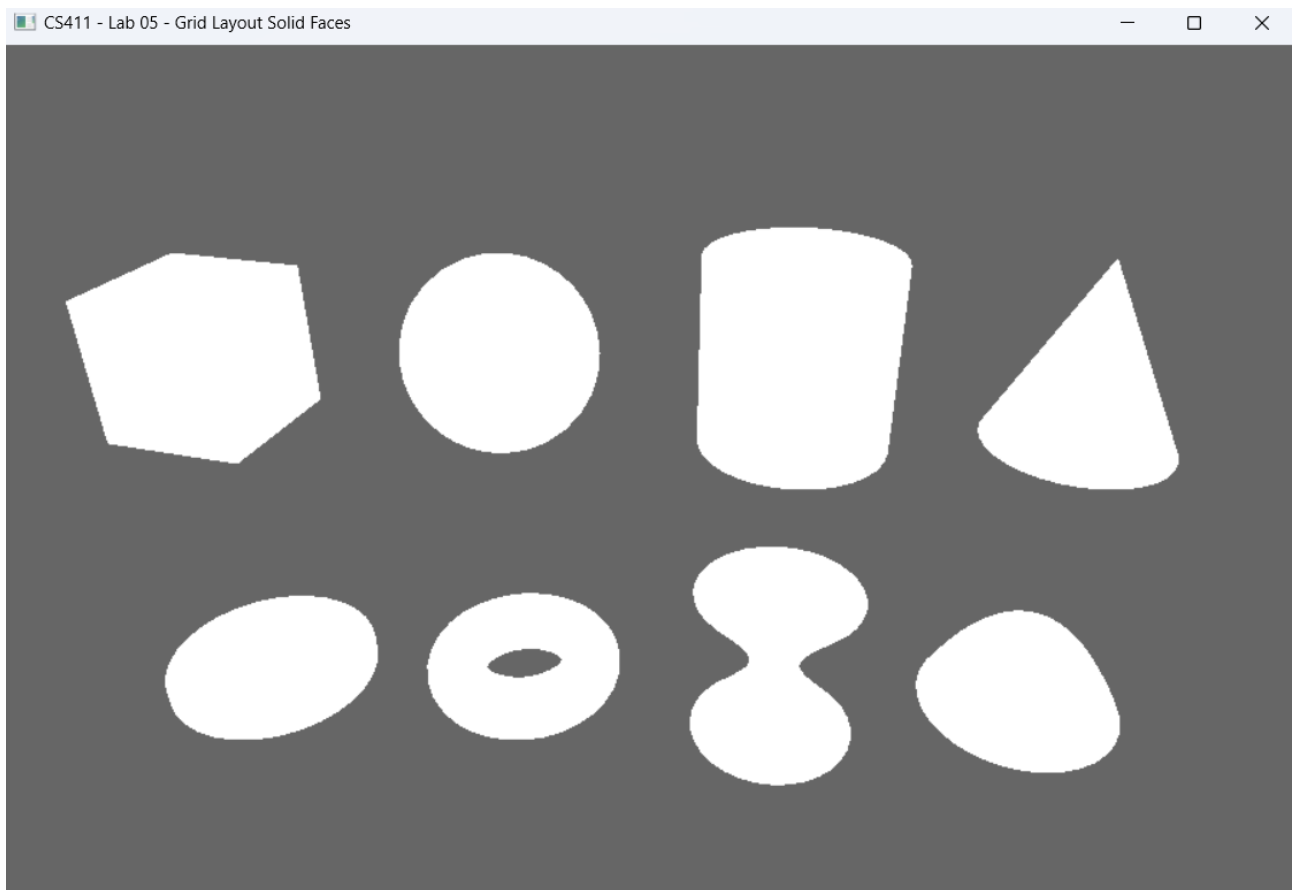


Figure 1: The screen after plotting all shapes, without texture mapping

5 Applying Clockwise Rotation

To fulfill the requirement that all shapes rotate on their own axis clockwise, we relied on a solitary global `angle` incremented in `glutTimerFunc`. We encapsulated each shape within `glPushMatrix()` and `glPopMatrix()`, shifted them into the grid using `glTranslatef`, and applied `glRotatef(-angle, ...)`. The negative angle forces a clockwise spin.

An important geometric override was necessary for the Torus (Ring). If we simply tilt it 90 degrees onto its side to face the camera, rotating on the Y-axis becomes visually unnoticeable. We fixed this by rotating it on its local Z-axis *after* the tilt:

```
glPushMatrix();
    glTranslatef(-1.5f, -2.0f, 0.0f);
    glRotatef(-90.0f, 1.0f, 0.0f, 0.0f); // Tilt flat
    glRotatef(-angle, 0.0f, 0.0f, 1.0f); // Spin clockwise around central hole
    // Draw Torus
glPopMatrix();
```

6 Texture Mapping Constraints

We expanded the basic SOIL loading block to handle 8 images loop-wise. Crucially, BMP images that are not cleanly powered by two often suffer from pixel unpack alignment skews. We remedied this by explicitly defining a 1-byte pixel unpack alignment:

```
glPixelStorei(GL_UNPACK_ALIGNMENT, 1);
```

Furthermore, mapping textures to complex topologies like the Torus requires projecting coordinates procedurally since freeglut primitives lack inherent UV bindings. To ensure the texture "sticks" and rotates precisely in tandem with the physical ring shape—rather than sliding behind a static camera projection—we forced `GL_OBJECT_LINEAR` mappings over the default eye-coordinate generation:

```
glEnable(GL_TEXTURE_GEN_S);
glEnable(GL_TEXTURE_GEN_T);
glTexGeni(GL_S, GL_TEXTURE_GEN_MODE, GL_OBJECT_LINEAR);
glTexGeni(GL_T, GL_TEXTURE_GEN_MODE, GL_OBJECT_LINEAR);
glutSolidTorus(0.35, 0.8, 32, 32);
glDisable(GL_TEXTURE_GEN_S);
glDisable(GL_TEXTURE_GEN_T);
```

Here is the final result:

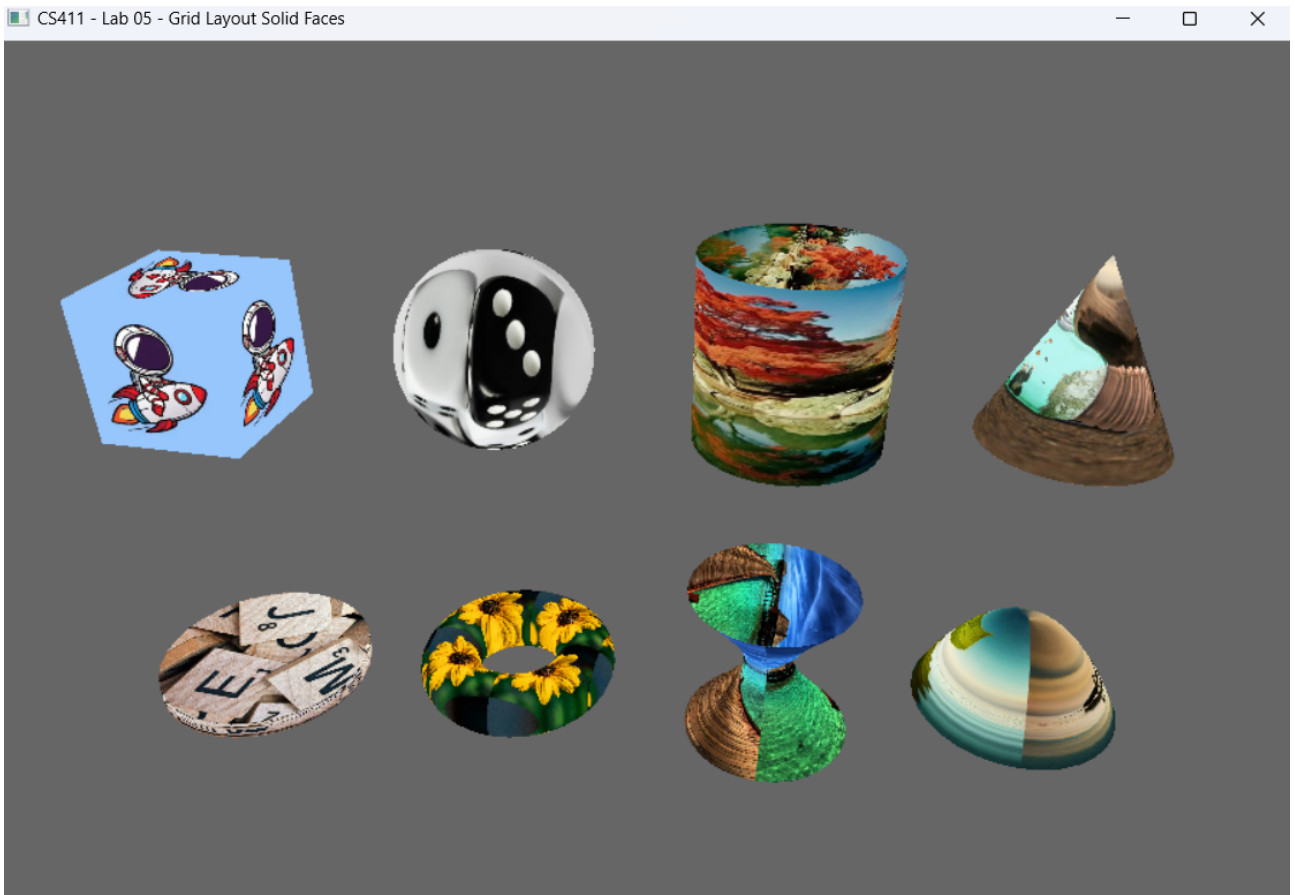


Figure 2: Main screen after applying the text mapping

7 Conclusion

Through this laboratory exercise, we have successfully developed a comprehensive 3D graphics application that fulfills all standard and custom rendering requirements. By implementing advanced parametric surfaces such as Hyperboloids and Paraboloids, we demonstrated the flexibility of the fixed-function OpenGL pipeline beyond standard library primitives.

The integration of independent per-face texturing and automated coordinate generation for complex shapes like the Torus allowed for a high degree of visual fidelity. Furthermore, the use of isolated matrix stacks ensured that each object maintained an independent clockwise rotation, providing a dynamic and synchronized animation across the grid. Ultimately, this project provided deep practical insights into coordinate systems, texture alignment, and the mathematical foundations of 3D mesh construction.